Deep Learning for Time Series: LSST Classification (id 9)

Paul Lemoine Vandermoere [1] **Baptiste Meresse** [1]

Abstract

In this report we present what we did for the project of Deep Learning for Time Series. We chose setting 2 and therefore tried to train a model on forecasting and then adapt it to classification. In order to be able to judge our results with this structure, we implemented a baseline that serves as reference for the score we get. Our report will consist of 4 parts. First, we will present the dataset and the problem we try to solve. Then, we will talk about the baseline we chose and justify its value as a "simple" solution. In a third time, we will explain how we trained a competitor to the baseline, relying on a pretraining for a forecasting task followed by a finetuning for classification. Finally, we will present our results and talk about the limits and possible improvements of our approach. The link for the repository is https://github.com/plemvan/Time-series-classification

1. Dataset and problematic

We use the data from the LSST dataset (Large Synoptic Survey Telescope). The data are simulated according to physics, with noise and blanks to respect the logic of the image capture. In this context, each data consists in the observation of a star in the sky for many nights. Each picture has 6 canals corresponding to the intensity in different parts of the light specter (from ultraviolet to infrared). In practice, the dataset we work on in a normalized version of the original one, in which each data has exactly 36 "pictures" taken on different night. We have about 2500 data for each of the train and test part.

As the LSST camera provides terras of data, we need to have some proper classification algorithm to know what the star is among 14 categories which consist of several galactic and extragalactic objects. Our goal is therefore to classify each objet given the different values of intensity we saw over the nights of observation. It is to note that descriptive statistics shows the classes aren't well balance: class 90 represents 31% of the dataset while class 53 represents only 0.3%. We therefore may need to introduce metrics "by class" in order to see if the rare one are well detected. As explained before, we will work in two times, first providing a baseline and then training an algorithm on forecasting to then finetune it on classification.

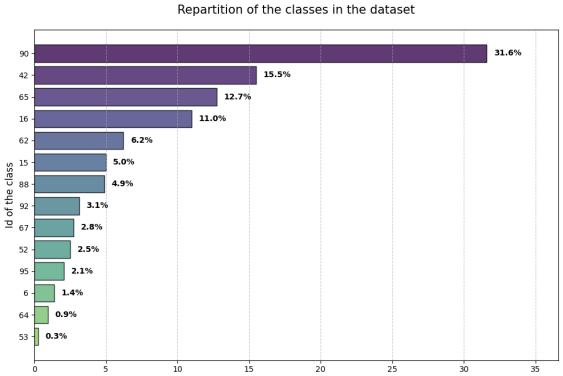

Figure 1. Repartition of the data among the different classes

2. Chosen baseline

For our baseline, we decided to rely on ROCKET (RandOm Convolutional KErnel Transform) (Dempster et al., 2020). This model uses a given number of random convolutional kernels (by default 10 000) to construct an embedding of our data. Each convolution is a 1×1 kernel sliding over time while looking at the different light filters. The random part comes from the fact that they don't all look at the 6 canals, but only to a random subset of them. It then outputs 2 different values based on the result of the convolution: the maximum value, and the number of positive values. This therefore result in a vector of length $20000 = n_{kernel} \times 2$. To get a classification out of this feature vector, we use a ridge classifier accounting that the penalization term helps to keep the model simple. It is to note that there is no deep learning here: we only get a diverse representation of our data with 10000 random kernel, and then try to find the best linear combination of the representation given by the kernels to predict the class.

Table 1 details the baseline performance across all 14 astronomical classes. While the model achieves an overall accuracy of 54%, a closer inspection of the support column reveals a severe class imbalance within the LSST dataset (see 5, e.g., class 90 contains 777 samples, whereas class 53 has only 7). Because global accuracy and weighted averages

	Precision	Recall	F1-Score	Support
accuracy			0.54	2466
macro avg	0.36	0.31	0.32	2466
weighted avg	0.51	0.54	0.52	2466

Table 1. Classification Report on LSST Dataset (ROCKET)

are heavily biased toward the majority classes, they mask the model's critical failure to detect rare astronomical events, as highlighted by the F1-scores equals to 0 for classes 53 and 64.

Consequently, for the remainder of this study, we will move away from evaluating overall accuracy and individual class scores. Instead, our primary evaluation metric will be the macro average F1-score (currently at 0.32). By computing the unweighted mean of the per-class F1-scores, the macro average ensures that minority classes are given the exact same importance as the most common ones.

3. Competitor architecture

Our methodology is anchored in the transfer learning paradigm for time series (Zerveas et al., 2021). We hypothesize that an encoder optimized to reconstruct the future horizon of a signal develops a deep semantic understanding that is highly transferable to discriminative tasks, such as the astronomical classification of the LSST dataset.

Our proposed model is trained in two steps. We first pre-train it on a forecasting task to build a foundation model, which we then fine-tune for the classification of the LSST dataset. For the pre-training phase, we utilize the ETTh1 (Electricity Transformer Temperature) dataset (Zhou et al., 2021). The "h" denotes hourly data, and the dataset contains 7 variables (including Oil Temperature, our target variable) with over 17,000 samples. Our objective is for the pre-trained model to learn the fundamental dynamics of time series well enough to generalize to a new dataset and a different task. Specifically, we use a look-back window of 96 hours to predict the next 24 hours in a single step.

A major challenge in this transfer learning setup is the structural discrepancy between the datasets: ETTh1 consists of 120-hour sequences with 7 variables, while LSST consists of 36-night sequences with 6 light filters. To bridge this dimensional gap, we adopted a channel-independent approach. Rather than forcing the model to learn cross-channel correlations, we treat each multivariate signal as a set of independent univariate time series. While this seemingly sets limits by ignoring the links between highly correlated channels, it is a necessary and highly beneficial trade-off. As demonstrated by recent advances in time series representation learning (Nie et al., 2023), channel independence significantly mitigates overfitting and effectively acts as data augmentation by virtually multiplying the training data size by the number of channels. Most importantly, it makes our architecture agnostic to the number of input variables, enabling seamless transfer from ETTh1 to LSST.

Concretely, regarding the architecture, we start from an input tensor of shape $batch_size \times n_channels \times sequence_length$. We reshape this into $(batch_size \times n_channels) \times 1 \times sequence_length$, treating each channel as a distinct batch item. This input is passed through three blocks consisting of 1D Convolutions, Batch Normalization, ReLU activations, and Max Pooling, concluding with an Adaptive Average Pooling layer. Through these layers, the hidden dimension expands from 1 to 32, 64 and finally 128, while the time axis is iteratively halved. Finally, we append a task-specific head (forecasting or classification) that processes this learned representation. This head consists of two fully connected layers: the first halves the hidden dimension, and the second outputs a tensor matching either the forecasting horizon or the number of target classes. In order to avoid overfitting, some dropout was present after the first fully connected layer (dropout = 0.4).

Table 2 summarizes the overall performance of our proposed approach, highlighting a significant improvement in the macro F1-score (see 5 for the detailed results).

	Precision	Recall	F1-Score	Support
accuracy			0.61	2466
macro avg	0.62	0.52	0.54	2466
weighted avg	0.63	0.61	0.60	2466

Table 2. Classification Report on LSST Dataset (Transfer learning)

4. Difficulties faced, solutions and refinements

4.1. Not overwritting the pre-training

The main goal and difficulty in this approach was to find the right balancing between pre-training and finetuning that make the model use the pre-trained encoder to improve the classification result. In fact, with unbalanced results, we risk the pre-training being useless and entirely undone or overwritten by the finetuning. In our first approach, where we just trained successively on both task, this is what happened. In fact, the pre-training was entirely overwritten by the finetuning, and we were getting the same results for both a pre-trained model and a model initialized with random weights. What we did to solve this was to improve our finetuning. We first froze the encoder for a given number of epochs in order to make the classification head learn without being able to change the representation it is given. We then unfroze the weights of the encoder and continued the training, but with a lower learning rate in order not to overfit. In this set up, the comparison between the random starting

weights and the pre-trained ones is clear.

4.2. The overfitting problem

Following the previous issue, another problem is potential overfitting on LSST dataset. In fact it contains a relatively small amount of data (2500) and a model too complex immediately leads to overfitting. We also believed the dataset was too little for us to rely on transformers architectures, which generally work better on large quantity of data. We therefore tried two different options of convolutional neural networks: with 2 or 3 layers. In practice, 3 layers worked better but required a smaller learning rate to not overfit too quickly on the dataset. Our way to detect overfitting was to add a validation loss on top of the training loss. The real indicator of our model improvement was the validation loss, as the model never saw the data from the validation set: if the training loss decrease but the validation loss doesn't anymore, it basically means that we don't learn anything useful anymore and are just overfitting.

4.3. Class imbalance and reproducibility

As some classes had very few data (the least populated star class only had 7), we had to change the way we updated the weight for our model to not just totally ignore those classes, which result in a good accuracy, but not macro f1-score. We tried to solve this problem by changing the value of the learning rate when updating the weigth according to the data: the more the class contains data, the less the update weigth will be and inversely. At first we just wanted the learning rate to be proportional to the number of data in the class, but this broke the whole model as we changed the weigths a lot only for a samples. We therefore made the learning rate be proportional to the square root of the number of data in the class.

To ensure strict reproducibility and mitigate variance caused by random weight initialization, we fixed the random seeds across our environment. Furthermore, to prevent overfitting and avoid late-stage training instability, we do not rely on the final epoch's model. Instead, we implemented model checkpointing to systematically save and evaluate the model that achieved the lowest validation loss during training.

4.4. Results

The detailed class-level performance is illustrated in the normalized confusion matrices in Figure 2. While the baseline model completely fails to identify minority classes, our architecture demonstrates a much more balanced predictive capability, proving that the model has learned robust temporal representations rather than merely exploiting the dataset's class imbalance.

5. Conclusion and possible improvements

The previous sections underlines the importance of finding the right parameters that give the good balance between pretraining and finetuning, training enought and not overfitting, optimizing the encoder without learning it from scratch. We therefore believe that a continuation of this project would be to potentially select a few parameters and do a grid search on them to try to better understand how the results are linked to them.

Additionally, our architecture relying on convolution layers for the encoder keeps a strict "temporal" vision of the data. By pre-training on forecasting, we therefore may have a representation that is unbalanced between start and end, as the last part of the temporal serie is more likely to give useful information for the prediction. Exploring symmetric pre-training tasks could help capture the entire time series equally and be a way to improve pre-training impact.

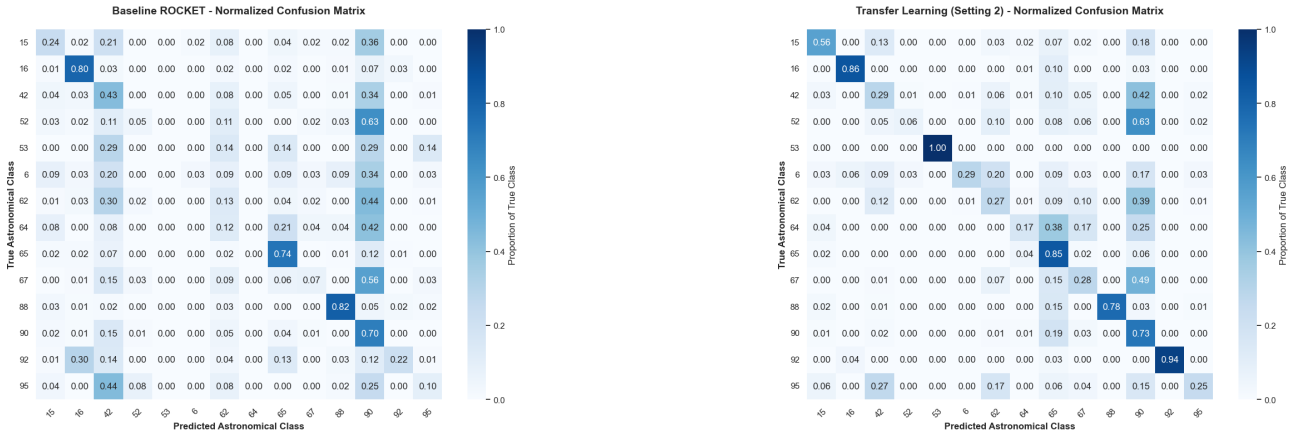

Figure 2. Visual comparison of the normalized confusion matrices on the LSST test set.

References

Dempster, A., Petitjean, F., and Webb, G. I. Rocket: exceptionally fast and accurate time series classification using random convolutional kernels. 2020.

Nie, Y., Nguyen, N. H., Sinthong, P., and Kalagnanam, J. A time series is worth 64 words: Long-term forecasting with transformers. 2023.

Zerveas, G., Jayaraman, S., Patel, D., Bhamidipaty, A., and Eickhoff, C. A transformer-based framework for multivariate time series representation learning. 2021.

Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., and Zhang, W. Informer: Beyond efficient transformer for long sequence time-series forecasting. 2021.

Appendix

This appendix provides the comprehensive, per-class evaluation metrics for both our baseline and the fine-tuned Transfer Learning model.

	ROCKET			**Transfer Learning**			
Class	**Precision**	**Recall**	**F1-Score**	**Precision**	**Recall**	**F1-Score**	**Support**
15	0.34	0.24	0.28	0.67	0.56	0.61	124
16	0.78	0.80	0.79	0.97	0.86	0.91	270
42	0.37	0.43	0.40	0.61	0.29	0.39	382
52	0.12	0.05	0.07	0.36	0.06	0.11	63
53	0.00	0.00	0.00	1.00	1.00	1.00	7
6	0.20	0.03	0.05	0.77	0.29	0.42	35
62	0.14	0.13	0.14	0.39	0.27	0.32	153
64	0.00	0.00	0.00	0.11	0.17	0.13	24
65	0.72	0.74	0.73	0.48	0.85	0.61	313
67	0.23	0.07	0.11	0.20	0.28	0.23	68
88	0.81	0.82	0.81	0.99	0.78	0.87	121
90	0.56	0.70	0.62	0.61	0.73	0.66	777
92	0.50	0.22	0.31	0.99	0.94	0.96	77
95	0.26	0.10	0.14	0.52	0.25	0.34	52
accuracy			0.54			0.61	2466
macro avg	0.36	0.31	0.32	0.62	0.52	0.54	2466
weighted avg	0.51	0.54	0.52	0.63	0.61	0.60	2466

Table 3. Class-by-class comparison between the ROCKET baseline and our model.